

Lab Sheet # L10

Instructions:

- Submit *.sql file, named as Lab10_6XXXXXX.sql to LAs or instructors prior to My Courses.
- If students cannot finish within the class time, students can submit until the beginning of the next class.

Self-study:

<https://www.mysqltutorial.org/mysql-stored-procedure/getting-started-with-mysql-stored-procedures/>

<https://www.mysqltutorial.org/mysql-stored-procedure/alter-stored-procedure/>

<https://www.mysqltutorial.org/mysql-stored-procedure/mysql-stored-function/>

<https://www.mysqltutorial.org/mysql-stored-procedure/mysql-if-statement/>

<https://www.mysqltutorial.org/mysql-stored-procedure/mysql-case-statement/>

Today's Task

0. Database Setup:

Import the database by executing the **PremierProducts.sql** and

Data_PremierProducts.sql files. (Same as Lab9)

Easy Task: basic stored procedure + function

1. Create procedure **sp_list_customers()** to show all customers.

Output columns: CustomerNum | CustomerName | City | State | Balance | CreditLine |

RepNum

25 • **CALL** sp_list_customers();

CustomerNum	CustomerName	City	State
148	Al's Appliance and Sport	Fillmore	FL
282	Brookings Direct	Grove	FL
356	Ferguson's	Northfield	FL
408	The Everything Shop	Crystal	FL

2. Create procedure **sp_list_parts()** to show all parts.

Output columns: PartNum | Description | OnHand | Class | Warehouse | Price

40 • **CALL** sp_list_parts();

PartNum	Description	OnHand	Class	Warehouse	Price
AT94	Iron	50	HW	3	24.95
BV06	Home Gym	45	SG	2	794.95
CD52	Microwave Oven	32	AP	1	165.00
DL71	Cordless Drill	21	HW	3	129.95

3. Create procedure `sp_list_orders()` to show all orders.

Output columns: OrderNum | OrderDate | CustomerNum

```
55 • CALL sp_list_orders();
```

OrderNum	OrderDate	CustomerNum
21608	2010-10-20 00:00:00	148
21610	2010-10-20 00:00:00	356
21613	2010-10-21 00:00:00	408
21614	2010-10-21 00:00:00	282

4. Create procedure `sp_list_reps()` to show all reps.

Output columns: RepNum | FirstName | LastName | City | State | Commision | Rate

```
70 • CALL sp_list_reps();
```

RepNum	FirstName	LastName	City	State	Commision	Rate
20	Valerie	Kaiser	Grove	FL	20542.50	0.05
35	Richard	Hull	Sheldon	FL	39216.00	0.07
65	Juan	Perez	Fillmore	FL	23487.00	0.05

5. Create procedure `sp_list_orderline()` to show all order lines.

Output columns: OrderNum | PartNum | NumOrdered | QuotedPrice

```
85 • CALL sp_list_orderline();
```

OrderNum	PartNum	NumOrdered	QuotedPrice
21608	AT94	11	21.95
21610	DR93	1	495.00
21610	DW11	1	399.99
21613	KL62	4	329.95

6. Create procedure `sp_list_currentorders()` to show all rows from `currentorders`.

Output columns: CustomerName | OrderNum | PartNum | Description | NumOrdered |

QuotedPrice | Warehouse | RepNum

```
100 • CALL sp_list_currentorders();
```

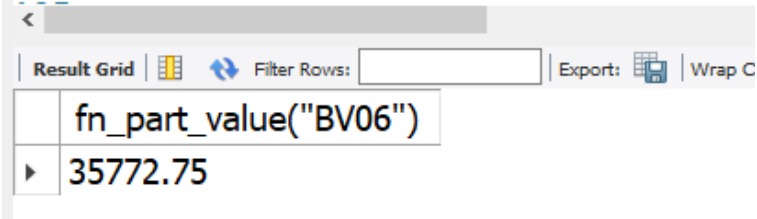
CustomerName	OrderNum	PartNum	Description
Al's Appliance and Sport	21608	AT94	Iron
Al's Appliance and Sport	21619	DR93	Gas Range
Brookings Direct	21614	KT03	Dishwasher
Ferguson's	21610	DR93	Gas Range

7. Create function `fn_part_value(p_partnum VARCHAR(10))`

Return total inventory value of that part: `OnHand * Price`.

Returns: `DECIMAL(10,2)`

```
124 • SELECT fn_part_value("BV06");
```



The screenshot shows a SQL query result grid. The top bar includes a back arrow, a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap C' option. The table has two columns: the first column contains the function call `fn_part_value("BV06")`, and the second column contains the result `35772.75`.

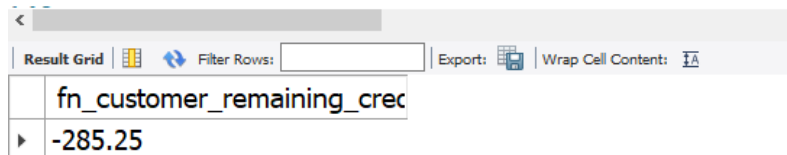
<code>fn_part_value("BV06")</code>
35772.75

-
8. Create function `fn_customer_remaining_credit(p_customernum INT)`

Return `CreditLine - Balance`.

Returns: `DECIMAL(10,2)`

```
148 • SELECT fn_customer_remaining_credit(408);
```



The screenshot shows a SQL query result grid. The top bar includes a back arrow, a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' option. The table has two columns: the first column contains the function call `fn_customer_remaining_cred`, and the second column contains the result `-285.25`.

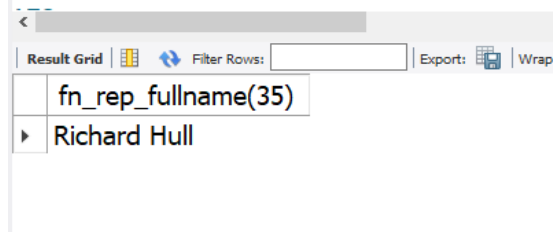
<code>fn_customer_remaining_cred</code>
-285.25

-
9. Create function `fn_rep_fullname(p_repnum INT)`

Return `"FirstName LastName"`.

Returns: `VARCHAR(200)`

```
172 • SELECT fn_rep_fullname(35);
```



The screenshot shows a SQL query result grid. The top bar includes a back arrow, a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap' option. The table has two columns: the first column contains the function call `fn_rep_fullname(35)`, and the second column contains the result `Richard Hull`.

<code>fn_rep_fullname(35)</code>
Richard Hull

10. Write a `SELECT` that shows each customer name and remaining credit using your function

`fn_customer_remaining_credit(CustomerNum)`. [From task 8]

Output columns: `CustomerName` | `RemainingCredit`

	CustomerName	RemainingCredit
▶	Al's Appliance and Sport	950.00
	Brookings Direct	9568.50
	Ferguson's	1715.00
	The Everything Shop	-285.25

Medium Task: IN/OUT/INOUT + variables + aggregation-based

11. Create procedure `sp_orders_by_customer(IN p_customernum INT)`

Show orders for that customer ordered by `OrderDate` DESC.

Output columns: `OrderNum` | `OrderDate` | `CustomerNum`

199 • `CALL sp_orders_by_customer(148);`

	OrderNum	OrderDate	CustomerNum
▶	21619	2010-10-23 00:00:00	148
	21608	2010-10-20 00:00:00	148

12. Create procedure `sp_parts_by_class(IN p_class VARCHAR(10))`

Show parts filtered by class, sort by `price` DESC.

Output columns: `PartNum` | `Description` | `Class` | `Price`

216 • `CALL sp_parts_by_class('AP');`

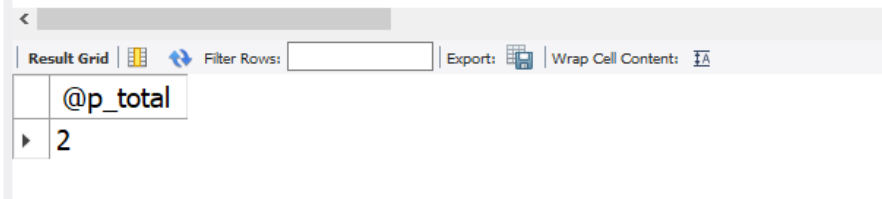
	PartNum	Description	Class	Price
▶	KT03	Dishwasher	AP	595.00
	DR93	Gas Range	AP	495.00
	DW11	Washer	AP	399.99
	KL62	Dryer	AP	349.95

13. Create procedure `sp_count_orders_by_customer(IN p_customernum INT, OUT p_total INT)`

Count how many orders the customer has, store into `p_total`.

Output: user will call and then `SELECT p_total`.

```
235 • CALL sp_count_orders_by_customer(148, @p_total);
236 • SELECT @p_total;
```



The screenshot shows the SQL Server Enterprise Manager interface. At the top, there's a toolbar with 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with one column labeled '@p_total' and one row containing the value '2'.

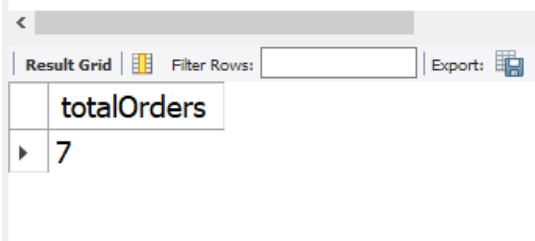
@p_total
2

14. Create procedure `sp_total_orders()`

Use a local variable (`DECLARE`) to store total number of rows in orders, then return it as a `SELECT` result.

Output columns: `totalOrders`

```
255 • CALL sp_total_orders();
```



The screenshot shows the SQL Server Enterprise Manager interface. At the top, there's a toolbar with 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with one column labeled 'totalOrders' and one row containing the value '7'.

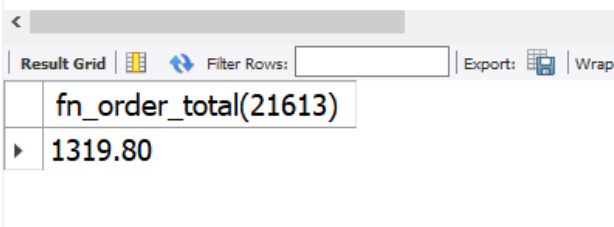
totalOrders
7

15. Create function `fn_order_total(p_ordernum INT)`

Return `SUM(NumOrdered * QuotedPrice)` for that order from `orderline`.

Returns: `DECIMAL(10,2)`

```
277 • SELECT fn_order_total(21613);
```



The screenshot shows the SQL Server Enterprise Manager interface. At the top, there's a toolbar with 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with one column labeled 'fn_order_total(21613)' and one row containing the value '1319.80'.

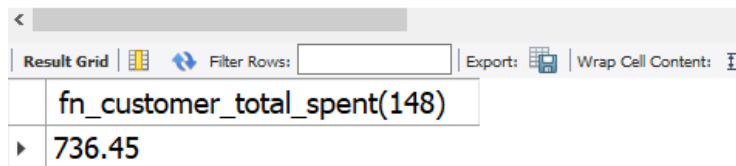
fn_order_total(21613)
1319.80

16. Create function `fn_customer_total_spent(p_customernum INT)`

Return total spent by that customer across all orders (`join orders + orderline`).

Returns: `DECIMAL(10,2)`

301 • `SELECT fn_customer_total_spent(148);`



The screenshot shows a SQL query result grid. The first row contains the text `fn_customer_total_spent(148)`. The second row contains the value `736.45`. The grid has a toolbar at the top with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'.

<code>fn_customer_total_spent(148)</code>
736.45

17. Write a `SELECT` that lists `OrderNum`, `OrderDate`, and `total amount` using `fn_order_total(OrderNum)` for all orders.

Output columns: `OrderNum` | `OrderDate` | `TotalAmount`

	<code>OrderNum</code>	<code>OrderDate</code>	<code>TotalAmount</code>
▶	21608	2010-10-20 00:00:00	241.45
	21610	2010-10-20 00:00:00	894.99
	21613	2010-10-21 00:00:00	1319.80
	21614	2010-10-21 00:00:00	1190.00
	21617	2010-10-22 00:00:00	2190.00

Hard Task: `INOUT` + variables + multi-step logic + `IF/ELSE` logic

18. Create procedure `sp_apply_credit_charge(IN p_customernum INT, IN p_charge DECIMAL(10,2), INOUT p_new_balance DECIMAL(10,2))`

Rules: Read current Balance into a local variable

Set `p_new_balance = Balance + p_charge`

Return `p_new_balance` (via `INOUT`)

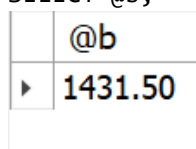
Also `SELECT p_new_balance AS NewBalance`

Expected use example (student must write `CALL` too):

`SET @b = 0;`

`CALL sp_apply_credit_charge(?, ?, @b);`

`SELECT @b;`



The screenshot shows a SQL query result grid. The first row contains the text `@b`. The second row contains the value `1431.50`.

<code>@b</code>
1431.50

19. Create procedure `sp_best_customer(OUT p_customernum INT, OUT p_customername VARCHAR(255), OUT p_total DECIMAL(10,2))`

Definition:

“Best customer” = highest total spent ($\text{SUM}(\text{NumOrdered} * \text{QuotedPrice})$)

Use join `customer` -> `orders` -> `orderline`

Store results into OUT params

Output: user calls and then selects the OUT variables.

	@a_customernum	@a_customername	@a_total
▶	608	Johnson's Department Store	4769.90

20. Create function `fn_credit_level(p_remaining DECIMAL(10,2))` that returns:

PLATINUM if `remaining` > 50000

GOLD if `remaining` between 10000 and 50000 (inclusive)

SILVER if `remaining` < 10000

Then write a `SELECT` using:

`fn_credit_level(fn_customer_remaining_credit(CustomerNum))`

Output columns: `CustomerName` | `RemainingCredit` | `CreditLevel`

	CustomerName	RemainingCredit	CreditLevel
▶	Al's Appliance and Sport	950.00	SILVER
	Brookings Direct	9568.50	SILVER
	Ferguson's	1715.00	SILVER
	The Everything Shop	-285.25	SILVER
	Bargains Galore	6588.00	SILVER